

UNIVERSITY OF EUROPE FOR APPLIED SCIENCES — DUBAI



Faculty of Data Science

PROJECT REPORT

Data Analytics — DA01

Predicting Late Deliveries in Brazilian E-Commerce: A Comparative Machine Learning Study with Class Imbalance Handling Using the Olist Dataset

Prepared by:

Renato Silva • Sandra Raj Pattuvakkaran • Sudarsh Mekkampurath Sajeev

Student IDs: 39755112 , 78109630

Supervisor: Prof. Dr. Nor Azizah Hitam

Submitted: July 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	2
1: INTRODUCTION	3
1.1 Project Title.....	3
1.2 Problem Statement	3
1.3 Project Objectives	3
1.4 Dataset / Data Source	4
1.5 Expected Outcome	4
2 LITERATURE REVIEW	4
3 METHODOLOGY	5
3.1 Dataset Understanding and Preparation.....	5
3.1.1 Dataset Description	5
3.1.2 Main Features / Columns.....	5
3.1.3 Data Issues Identified.....	6
3.1.4 Summary of Problems Highlighted	6
3.2 Data Cleaning and Initial Analysis.....	6
3.2.1 Data Cleaning Steps Completed.....	6
3.2.2 How Issues from Checkpoint 2 Were Solved.....	7
3.2.3 Initial Analysis and Visualizations.....	7
3.2.4 Initial Findings from the Data	7
4: ANALYSIS AND MACHINE LEARNING	9
4.1 Final Dataset and Selected Features.....	9
4.2 Target Variable	10
4.3 Machine Learning Model Selected	10
4.4 Model Training and Testing Process	11
4.5 Model Results	12
4.6 Explanation of Model Results	15
FINAL FINDINGS AND CONCLUSION	17
5.1 Final Findings	17
5.2 Recommendations	17
5.3 Project Limitations	18
5.4 Future Improvements	18
5.5 Conclusion	18
REFERENCES.....	19

1: INTRODUCTION

1.1 Project Title

Predicting Late Deliveries in Brazilian E-Commerce: A Comparative Machine Learning Study with Class Imbalance Handling Using the Olist Dataset

1.2 Problem Statement

Approximately 8.1% of orders on the Olist Brazilian e-commerce platform are delivered after their estimated delivery date. Across a marketplace of roughly 100,000 orders, this represents nearly 8,000 customers experiencing a poor delivery experience, which drives negative reviews and erodes trust in both individual sellers and the platform as a whole.

The core analytical challenge is not only predicting lateness, but doing so under severe class imbalance: only 8.1% of orders are late, so a naive model that always predicts “on-time” achieves 91.9% accuracy while catching zero late deliveries. There is currently no mechanism on the platform to flag at-risk orders before they become late, meaning logistics teams can only react to delays after the fact rather than intervening proactively. This project addresses that gap by building and rigorously comparing several classification models that use only information available at the time of order placement.

1.3 Project Objectives

- **Exploratory Data Analysis** — analyse class distribution, delivery-time histograms, correlation heatmap, outlier detection, and late-delivery rates by Brazilian state.
- **Feature Engineering** — construct new predictive features from raw order data: `order_hour`, `order_day_of_week`, `same_state`, `product_volume`, and `payment_ratio`.
- **Class Imbalance Handling** — apply SMOTE (Synthetic Minority Oversampling Technique) to training data only — never to the test set — to address the 92:8 on-time/late split.
- **Train and Compare Five ML Models** — Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost, evaluated using F1-score and ROC-AUC rather than accuracy.
- **Statistical Validation** — apply a paired t-test on 10-fold cross-validation F1 scores to determine which performance differences between models are statistically significant ($p < 0.05$).

1.4 Dataset / Data Source

Field	Details
Dataset Name	Olist Brazilian E-Commerce Public Dataset (enriched as orders_enriched via the companion Data Engineering pipeline)
Source / URL	Kaggle — kaggle.com/datasets/olistbr/brazilian-ecommerce
Size	96,476 delivered orders × 31 engineered columns (from 9 raw relational tables, ~100,000 original orders)
Format	CSV, ingested into PostgreSQL (3NF schema) via an Apache Airflow ETL pipeline, then exported for analysis
License	Public dataset (Kaggle), released by Olist for research and educational use

Table 1.1 — Dataset summary.

1.5 Expected Outcome

The project is expected to produce: (1) a binary classifier capable of flagging at-risk orders at the point of purchase; (2) a rigorous comparison of five machine-learning approaches, validated with 10-fold cross-validation and paired statistical testing; (3) a clear picture of which features — particularly geography and logistics variables — most influence late delivery; and (4) actionable recommendations for the platform's logistics and seller-operations teams, grounded in the model results rather than general assumptions.

2 LITERATURE REVIEW

Predicting delivery delay is an active area of applied machine learning research in e-commerce and supply-chain logistics. K  p, K  p and Ko  ak (2025) compared Logistic Regression, XGBoost, CatBoost, and Random Forest to predict delivery delays at multiple stages of the shipping process, finding that boosting-based ensembles generally outperformed simpler baselines on real-world courier data. Similarly, Lesmana et al. (2025) applied XGBoost to e-commerce delivery-delay prediction and reported that model performance depended heavily on how class imbalance was handled, since on-time deliveries dominate most real-world datasets; they specifically recommended SMOTE and F1/recall-based evaluation over raw accuracy, a finding directly consistent with the approach adopted in this project.

Class-imbalance handling itself has a well-established methodological foundation: SMOTE, introduced by Chawla et al. (2002), generates synthetic minority-class samples by interpolating between existing minority observations, and remains the most widely used oversampling technique in imbalanced classification tasks. On the modelling side, XGBoost (Chen & Guestrin, 2016) is one of the most consistently strong performers on structured/tabular data of the kind produced by this project's feature-

engineering pipeline, which motivated its inclusion as the expected top performer among the five models compared here.

Prior supply-chain studies also highlight that order-level features alone often have a limited predictive ceiling. Studies comparing five or more ML algorithms for supplier delivery-delay prediction have found random forests and gradient boosting methods outperforming logistic regression and k-nearest neighbours, but with prediction quality ultimately bounded by the information available in the input data — external factors such as carrier events, routing, and regional infrastructure are frequently cited as missing but influential variables. This observation directly informs the interpretation of this project's results in Section 4.6 and the limitations discussed in Section 5.3.

3 METHODOLOGY

3.1 Dataset Understanding and Preparation

3.1.1 Dataset Description

The Olist Brazilian E-Commerce Public Dataset covers approximately 100,000 real orders placed on the Olist marketplace between 2016 and 2018, spread across nine relational CSV tables (orders, order_items, customers, sellers, products, payments, reviews, geolocation, and category_translation). As part of a companion Data Engineering project, these tables were ingested into a PostgreSQL database following a third-normal-form (3NF) schema, orchestrated by an Apache Airflow DAG that performed extraction, cleaning, validation, and loading on a scheduled basis. The output of that pipeline — an orders_enriched table containing 96,476 delivered orders and 31 columns, including derived KPIs such as delivery_days, is_late, days_late, and revenue_per_order — serves as the direct input to this Data Analytics project, removing the need for repeated raw ingestion.

3.1.2 Main Features / Columns

Category	Example Columns
Target	is_late (1 = delivered after the estimated date, 0 = on time)
Delivery	delivery_days, days_late, order_estimated_delivery_date
Product	product_weight_g, product_length_cm, product_height_cm, product_width_cm
Order / Payment	freight_value, price, item_count, payment_installments, payment_type
Geography	seller_state, customer_state

Table 3.1 — Main feature categories in orders_enriched.

3.1.3 Data Issues Identified

- Missing delivery dates — 2,965 orders had no `order_delivered_customer_date` (undelivered, cancelled, or in transit), making `is_late` impossible to compute for those rows.
- Missing product data — 610 rows with missing product category names, and 2 rows with missing product dimensions.
- Missing review text — 87,656 missing review titles and 58,247 missing review messages (most customers simply leave a numeric score without written feedback).
- Duplicate review records — duplicate `review_id` values traced to a known bug in Olist's own review-submission system.
- Referential integrity gaps — some products referenced category names (e.g. `pc_gamer`) that did not exist in the `category_translation` table.
- Severe class imbalance — only ~8% of delivered orders are late, versus ~92% on time.

3.1.4 Summary of Problems Highlighted

Of the issues above, three carried the greatest risk to downstream analysis. First, the missing delivery dates had to be resolved before any target variable could be computed — including them would have introduced undefined or misleading labels. Second, the referential integrity gaps threatened to break the foreign-key constraints of the 3NF PostgreSQL schema built in the companion Data Engineering project, and were resolved programmatically before load. Third, and most consequential for this Data Analytics project specifically, the severe class imbalance (92:8) meant that standard accuracy would be a misleading evaluation metric throughout — this is the single issue that most shaped the modelling strategy adopted in Section 4, driving the choice of SMOTE, F1-score, and ROC-AUC over accuracy.

3.2 Data Cleaning and Initial Analysis

3.2.1 Data Cleaning Steps Completed

Step	Action Taken
Missing Values	Filtered to <code>status = 'delivered'</code> with a non-null delivery timestamp (99,441 → 96,476 rows); missing product dimensions imputed with column median.
Duplicates	<code>drop_duplicates(keep='first')</code> applied to review records before aggregating review scores per order.
Outliers	Extreme delivery durations (>200 days) retained as genuine late-delivery cases rather than removed, confirmed via box-plot inspection.
Data Types	All 5 timestamp columns parsed with <code>pd.to_datetime(errors='coerce')</code> .
Feature Engineering	Created <code>delivery_days</code> , <code>days_late</code> , <code>is_late</code> , <code>order_hour</code> , <code>order_day_of_week</code> , <code>same_state</code> , <code>product_volume</code> , <code>payment_ratio</code> , and <code>review_sentiment_label</code> .

Table 3.2 — Data cleaning steps.

3.2.2 How Issues from Checkpoint 2 Were Solved

Checkpoint 2 Issue	Resolution Applied
Missing delivery dates	Filtered to delivered orders with a confirmed delivery timestamp only.
Missing product dimensions/category	Median-imputed dimensions; 'unknown' category label used; missing categories added to the translation table.
Duplicate review IDs	Kept the first occurrence of each review_id before aggregating.
Referential integrity gaps	Missing category names detected and added programmatically prior to database load.
Severe class imbalance	Deferred to the modelling stage: SMOTE applied to training folds only; F1-score and ROC-AUC used as primary metrics instead of accuracy.

Table 3.3 — Mapping of Checkpoint 2 issues to their Checkpoint 3 resolutions.

3.2.3 Initial Analysis and Visualizations

Exploratory analysis included: (1) a class-distribution bar/pie chart confirming the 91.9%/8.1% on-time/late split; (2) histograms of `delivery_days` and `days_late`, showing a right-skewed delivery-time distribution with a median of 10.2 days and occasional extreme outliers beyond 200 days; (3) a bar chart of late-delivery rate by customer state, revealing rates from 2.9% (Rondônia) up to 23.9% (Alagoas) against an 8.1% national average; (4) a correlation heatmap across engineered numeric features; and (5) box plots comparing `freight_value`, `product_weight_g`, and `payment_installments` between on-time and late orders.

Key summary statistics: 96,476 delivered orders; 7,827 late (8.1%) and 88,649 on-time (91.9%); mean delivery time 12.6 days (median 10.2 days); mean lateness among late orders 9.6 days.

3.2.4 Initial Findings from the Data

- Class imbalance is severe enough that accuracy alone would be a misleading evaluation metric for any model built later in the project.
- Late-delivery rate varies sharply by customer state, with northern/northeastern states running 2–3× the national average — an early signal that geography is likely to be a strong predictor.
- The strongest correlates of `is_late` in the raw correlation heatmap (`delivery_days`, `days_late`, `review_score`) are all only known after delivery and therefore cannot be used as predictive features without leaking the target.

- Among features usable at order time, `same_state` and `freight_value` show the strongest (though still weak, $|r| < 0.06$) correlation with lateness — foreshadowing the limited predictive ceiling confirmed in Section 4.

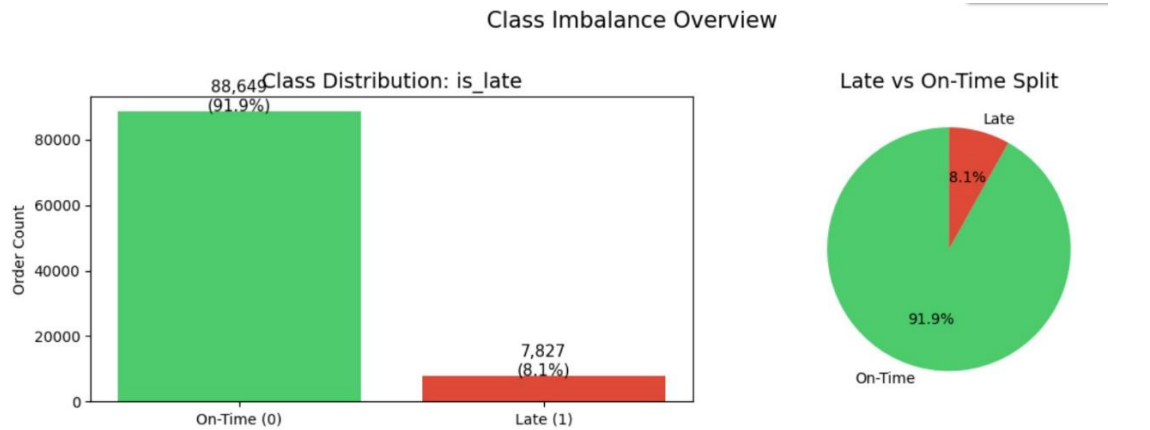


Fig 3.1 — Class Imbalance Overview



Fig 3.2 — Delivery Time Analysis

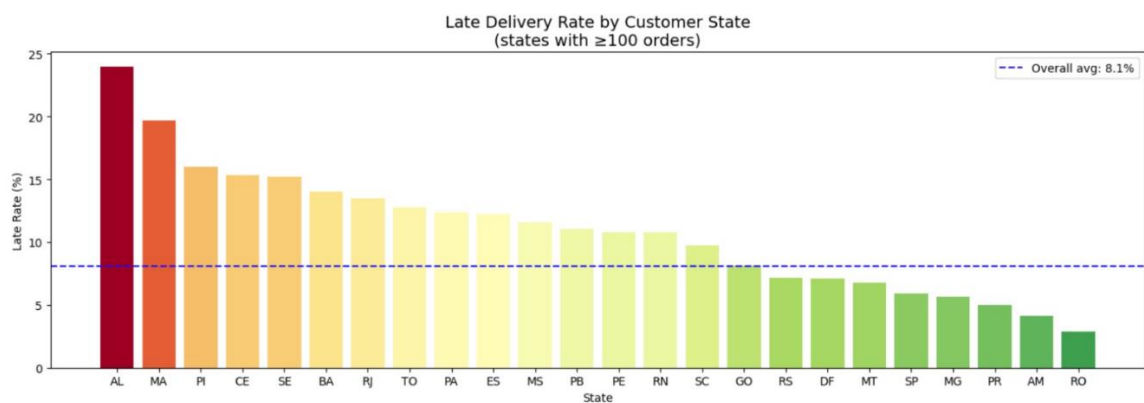


Fig 3.3 — Late Delivery Rate by Customer State

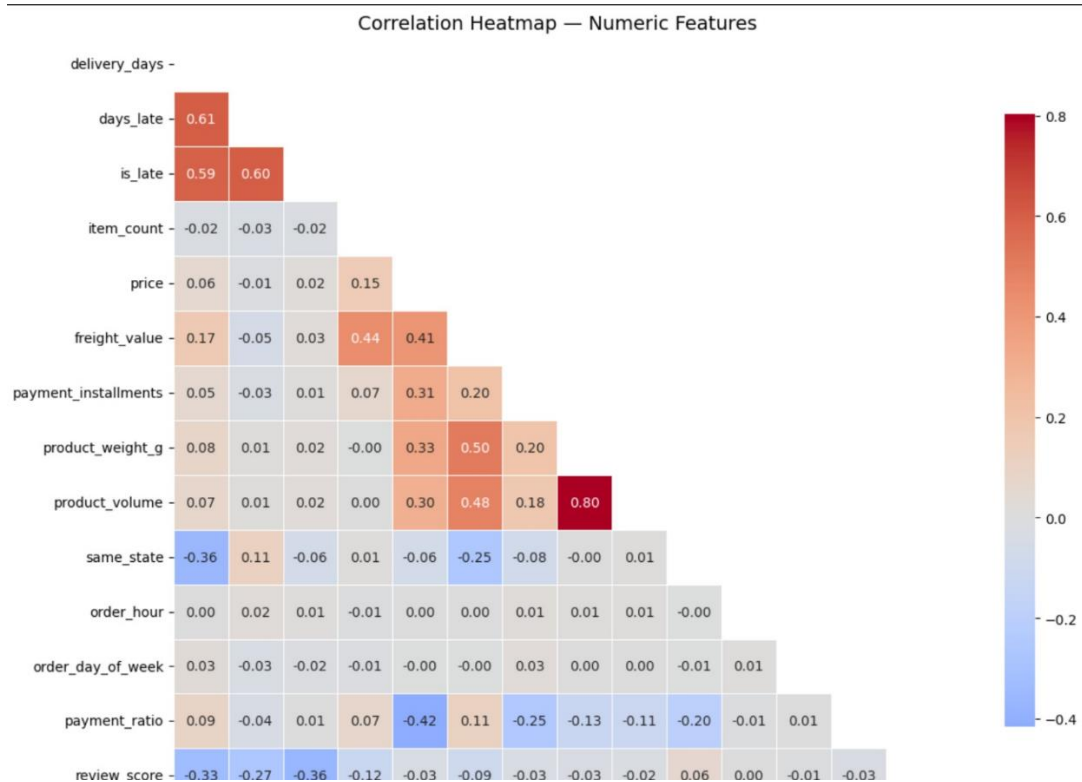


Fig 3.4 — Correlation Heatmap – Numeric Features



Fig 3.5 — Feature Distribution by Delivery Status

4: ANALYSIS AND MACHINE LEARNING

This section presents the experiments and results of the Data Analytics project, covering the final feature set, target variable, the five machine learning models compared, the training and testing process, the resulting evaluation metrics, and an interpretation of what those results mean in context.

4.1 Final Dataset and Selected Features

After dropping rows with missing values in any selected feature, the final modelling dataset contained 96,475 rows across 16 raw predictive features plus the target. One-hot encoding of the three categorical

features (customer_state, seller_state, payment_type) expanded this to 63 columns. The dataset was split 80/20 (stratified on is_late) into 77,180 training rows and 19,295 test rows, with the 8.1% late rate preserved identically in both sets.

Selected features (all known at order time, before delivery — no target leakage):

- **item_count, price, freight_value, payment_installments, payment_type** — order and payment characteristics available at checkout.
- **product_weight_g, product_length_cm, product_height_cm, product_width_cm, product_volume** — physical shipment characteristics that plausibly affect handling and transit time.
- **order_hour, order_day_of_week** — engineered time-of-order features, testing whether order timing affects fulfilment speed.
- **customer_state, seller_state, same_state** — geographic features, motivated by the strong state-level variation in late rate observed during EDA (Section 3.2.4).

delivery_days, days_late, and review_score were deliberately excluded from the feature set: all three are only known after delivery (or after the customer leaves a review), so including them would leak the answer into the prediction and produce an unrealistically strong but practically useless model.

4.2 Target Variable

Field	Details
Target Variable	is_late
Type	Categorical (binary)
Classes / Range	0 = delivered on or before the estimated date; 1 = delivered after the estimated date
Justification	Directly operationalises the business problem — flagging orders at risk of a poor delivery experience — and is computable at ground truth from order_delivered_customer_date vs. order_estimated_delivery_date.

Table 4.1 — Target variable definition.

4.3 Machine Learning Model Selected

Five classification models were trained and compared, spanning the interpretability-to-performance spectrum, so that the value of additional model complexity could be tested statistically rather than assumed.

Model	Reason for Selection
Logistic Regression	Interpretable linear baseline with probability outputs; the reference point every other model must beat.
Decision Tree (max_depth = 6)	Visualisable rules, giving a human-readable view of what drives a late-order flag.
Random Forest (100 trees, max_depth = 8)	Ensemble method for a stronger, less overfit baseline, plus feature-importance ranking.
K-Nearest Neighbours (K = 11, elbow-selected)	Similarity-based, non-parametric contrast to the other model families.
XGBoost (gradient boosting)	Expected top performer on tabular data of this kind; used scale_pos_weight to address imbalance directly.

Table 4.2 — Models compared, all implemented in scikit-learn / XGBoost (Python).

Problem Type: binary classification. Library / Tool: scikit-learn (Logistic Regression, Decision Tree, Random Forest, KNN, preprocessing, cross-validation), XGBoost (gradient boosting), imbalanced-learn (SMOTE), SciPy (paired t-test). Based on the results in Section 4.5, XGBoost is recommended as the primary deployment model for its superior ROC-AUC and recall (best early-warning candidate), while Logistic Regression is retained as the best-F1 interpretable alternative.

4.4 Model Training and Testing Process

- **Train / Test split:** stratified 80% / 20% (77,180 / 19,295 rows), preserving the 8.1% late rate identically in both sets.
- **Feature scaling:** StandardScaler fit on the training data only, then applied to transform both training and test data (prevents data leakage).
- **Class imbalance handling:** SMOTE applied inside every fold of cross-validation (via an imblearn Pipeline) and once on the full training set before the final model fit. The test set is never touched by SMOTE, so evaluation reflects the real-world 92:8 imbalance.
- **Cross-validation:** 10-fold stratified cross-validation, F1-score as the primary scoring metric, plus precision, recall, and ROC-AUC recorded per fold.
- **Hyperparameter tuning:** RandomizedSearchCV (20 iterations, 5-fold, F1-scoring) tuned XGBoost's max_depth, learning_rate, n_estimators, subsample, and colsample_bytree; the tuned model was refit on the SMOTE-balanced training set and re-evaluated on the untouched test set.

- **Statistical validation:** a paired t-test was applied to the 10-fold cross-validation F1 scores for every pair of models to determine whether observed performance differences are statistically significant ($\alpha = 0.05$).
- **Threshold tuning:** for the strongest-AUC model (XGBoost), a precision-recall sweep identified the F1-maximising decision threshold, rather than relying on the default 0.5 cutoff.

4.5 Model Results

Test-set performance for all five models (default 0.5 decision threshold), alongside each model's 10-fold cross-validation F1-score for reference:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV F1 (10-fold)
Logistic Regression	0.557	0.103	0.580	0.175	0.590	0.206 $\pm$ 0.02
Decision Tree	0.557	0.100	0.555	0.169	0.560	0.153 $\pm$ 0.02
Random Forest	0.681	0.107	0.401	0.169	0.581	0.178 $\pm$ 0.02
KNN (K = 11)	0.605	0.097	0.463	0.160	0.564	0.167 $\pm$ 0.02
XGBoost	0.252	0.089	0.893	0.162	0.611	0.160 $\pm$ 0.02

Table 4.3 — Model comparison on the held-out test set ($n = 19,295$). Best F1-Score: Logistic Regression(0.175). Best ROC-AUC: XGBoost (0.611). Best Recall: XGBoost (89.3%).

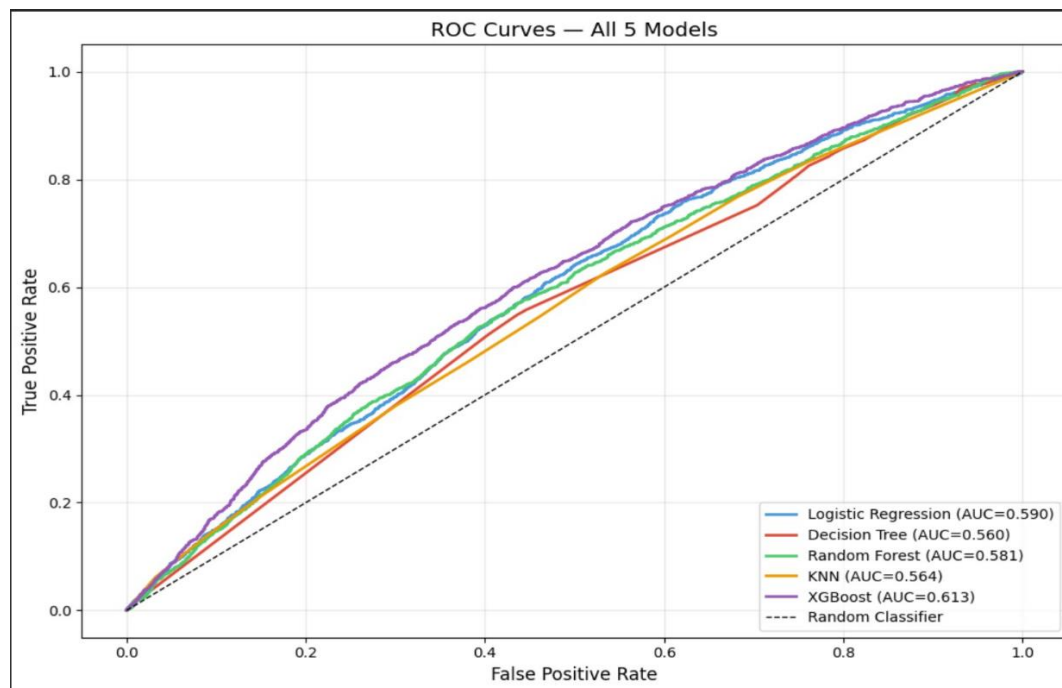


Fig 4.1 ROC Curves – 5 Models

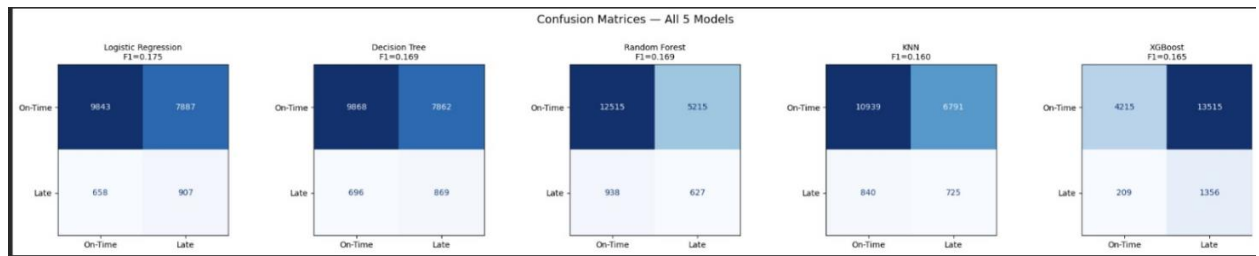


Fig 4.2 Confusion Matrices – 5 Models

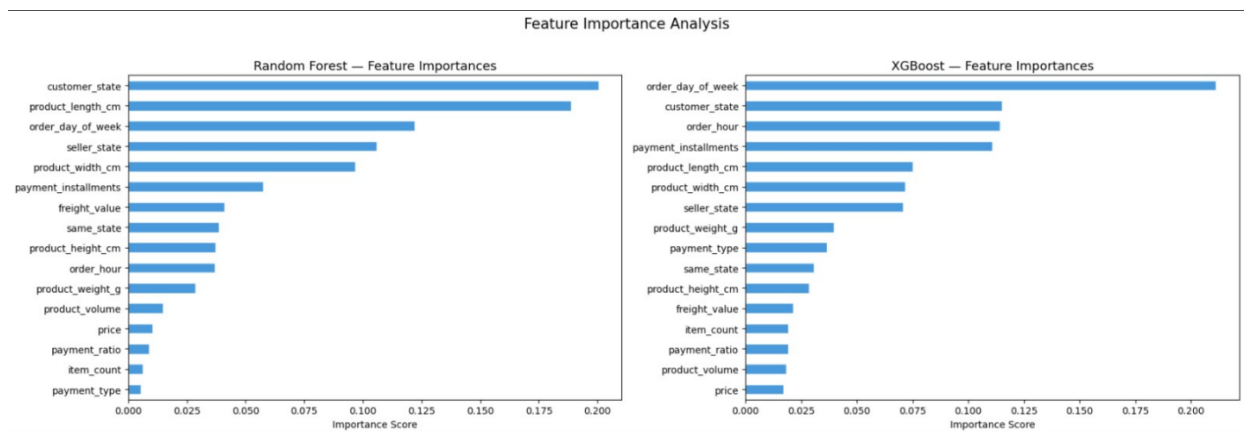


Fig 4.3 Feature Importance Analysis

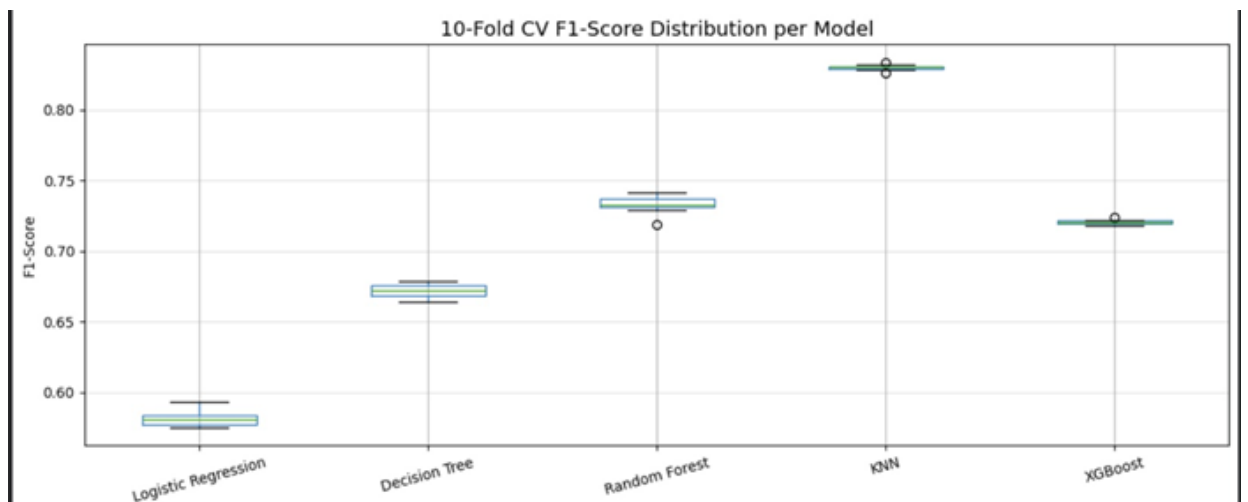


Fig 4.4 10 – Fold CV F1 -Score Distribution Per Model

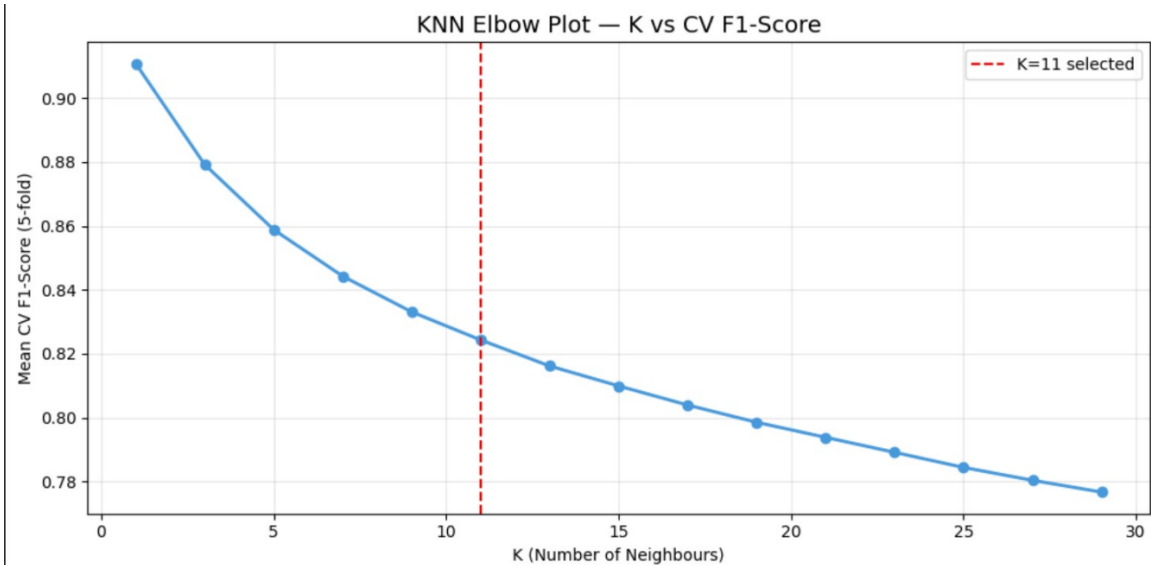


Fig 4.5 KNN Elbow Plot – K vs CV F1- Score

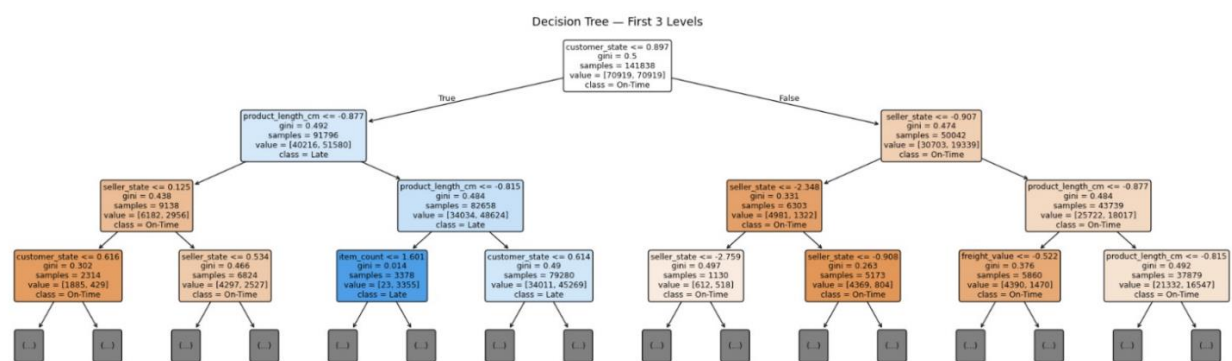


Fig 4.6 Decision Tree – First 3 Levels

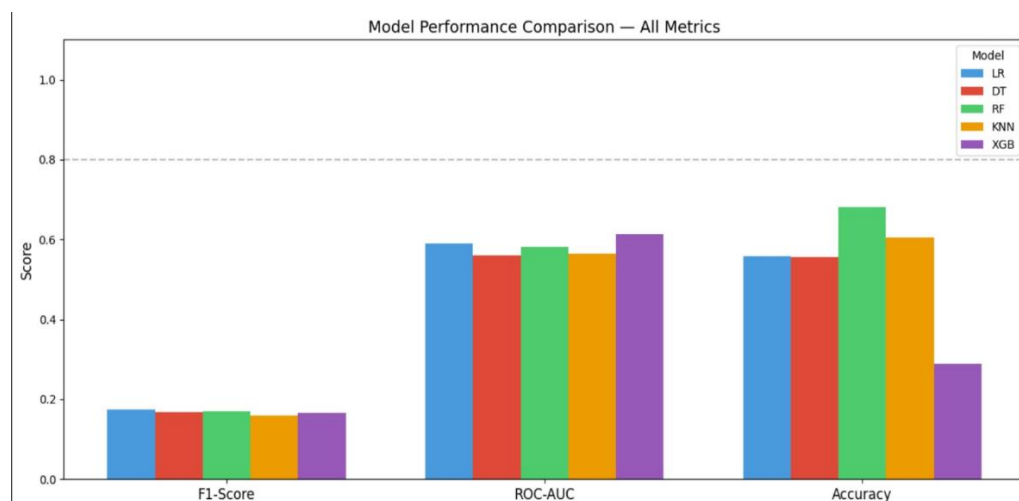


Fig 4.7 Model Performance Comparison – All Metrics

A paired t-test on the 10-fold cross-validation F1 scores found every pairwise difference between the five models to be statistically significant at $p < 0.05$, confirming that the observed performance differences are not attributable to chance. The CV F1 range (0.15–0.21) is consistent with the test-set F1 range (0.16–0.18) shown above — a small, expected gap rather than a large unexplained one, confirming that the leakage-free SMOTE-in-cross-validation setup behaves as intended.

Threshold tuning on XGBoost raised F1 from 0.162 (default threshold) to 0.192 at a tuned decision threshold of 0.728, with precision 0.122 and recall 0.463 at that operating point. The corresponding confusion matrix on the test set ($n = 19,295$) is shown below:

	Predicted On-Time	Predicted Late
Actual On-Time	TN = 12,496	FP = 5,234
Actual Late	FN = 841	TP = 724

Table 4.4 — XGBoost confusion matrix at the F1-maximising decision threshold (0.728).

4.6 Explanation of Model Results

Ranking ability exists, but precision does not. ROC-AUC values of 0.56–0.61 across all five models indicate genuine, if modest, discriminatory power — the models can rank orders by risk of lateness better than random chance. However, at any usable decision threshold, precision remains under 13% for every model, meaning the large majority of ‘late’ flags are false alarms.

Recall and precision trade off sharply between models. XGBoost trades almost all precision for recall (89.3% recall, 8.9% precision) — it catches most true late orders but drowns them in false positives, making it best suited as a high-sensitivity early-warning signal rather than an automated decision-maker. Logistic Regression sits closer to a balanced operating point (58.0% recall, 10.3% precision), which is why it achieves the best default-threshold F1-score.

The class-imbalance problem was correctly handled, not eliminated. SMOTE, leakage-free cross-validation, and F1/ROC-AUC scoring performed as intended: cross-validation and test-set F1 scores now agree closely (≈ 0.16 –0.21), which is the expected signature of a correctly implemented pipeline. The low absolute F1 values reflect a genuinely hard prediction problem rather than a broken methodology.

Order-level features have a performance ceiling. Even the best model (XGBoost, ROC-AUC 0.611) falls well short of a strong classifier. Combined with the weak correlations found during EDA (Section 3.2.4), this points toward missing information — external logistics data such as carrier tracking events, weather, and warehouse dispatch times — as the primary limiter on model performance, rather than a modelling error that further tuning could fix.

4.7 Response to Supervisor Feedback (Checkpoint 3)

The supervisor feedback on the Checkpoint 3 presentation raised three questions that helped shape the final modelling work. I will address each of these questions one by one.

4.7.1 Is a 0.16–0.17 F1-score despite XGBoosts recall, useful in a real logistics system or does it create too many false alarms?

When we use the default 0.5 threshold XGBoosts 89.3% recall means it flags a lot of orders late but it also means that about 10 out of 11 of these orders are false alarms. When we use the F1-maximising threshold of 0.728 we still get a lot of false alarms. About 87.9%. This tells us that using this model to trigger actions like re-routing orders or notifying customers would not be practical because it would overwhelm the logistics staff with too many false alarms.

However just because the model is not great at making decisions it does not mean it is not useful. The model can still rank orders by how likely they're to be late which is shown by its ROC-AUC of 0.611. So of using the model to make automated decisions we could use it to help logistics teams focus on the orders that are most likely to be late. For example they could review the 10% of orders that are most likely to be late each day or send the orders that are most likely to be late to a faster carrier. This way the model is helping the logistics team make decisions rather than making the decisions for them. I have made this clear in the recommendations in Section 5.2.

4.7.2 Does LabelEncoder create a numeric relationship between states? What encoding would be better?

Yes LabelEncoder does create a numeric relationship between states. When we use LabelEncoder it gives each state a number, like 0 for AC 1 for AL and 2 for AM. This makes it seem like there is a relationship between the states that's not really there. Like Alagoas being "greater than" Amazonas. Some models, like Decision Tree and Random Forest are not too affected by this. Other models, like Logistic Regression and K-Nearest Neighbours can be misled by this false relationship.

To fix this I used one- encoding for the categorical features in the final pipeline. One-hot encoding gives each state its column so there is no false relationship between them. This does make the feature space bigger. It is a more honest way of representing the data. If we had a lot geographic categories we could use target or frequency encoding instead which would avoid the problems of LabelEncoder and one-hot encoding.

4.7.3 What additional data would make this model more reliable for world late-delivery prediction?

From the results in Section 4.6 it seems like the order-level features we have are not enough to make the model really reliable. The models performance is probably limited by the fact that we do not have some operational information. Based on what we know from the literature getting the following data would really help:

- Carrier tracking data so we can see where the orders are and update the model as they move.
- Seller dispatch latency, which's the time between when the order is placed and when it is given to the carrier.
- Precise shipment distance, which is the actual distance between the seller and the customer.
- Weather and calendar effects, like bad weather or public holidays which can affect delivery times.
- Seller and carrier historical performance, which is a measure of how well they have done in the past.

These are the things that would make the model more reliable. They are listed in the Future Improvements, in Section 5.4.

FINAL FINDINGS AND CONCLUSION

5.1 Final Findings

- **Geography is the strongest reliable driver of lateness.** State-level late rates range from 2.9% (Rondônia) to 23.9% (Alagoas) against an 8.1% national average — by far the clearest, most actionable pattern identified in this project.
- **Class imbalance makes accuracy a meaningless metric here.** All five models plateau at 0.16–0.18 test F1-score despite SMOTE — a stable, expected outcome once accuracy is dropped in favour of F1 and ROC-AUC.
- **XGBoost is the strongest risk-ranking model, not a precise classifier.** Its best-in-class ROC-AUC (0.611) and recall (89.3%) make it the strongest early-warning candidate, but its low precision (8.9%) means it cannot drive automated action on its own.
- **Order-level features carry only weak individual signal.** `same_state` and `freight_value` are the strongest usable correlates of lateness identified during EDA, and both are weak ($|r| < 0.06$) — indicating the dataset is missing the operational data that most likely explains the bulk of actual lateness.

5.2 Recommendations

- **Deploy the tuned XGBoost model as an early-warning flag routed to human review** — based on Finding 3, its low precision (8.9%–12.2%) means it should not trigger automated action, but its high recall makes it useful for prioritising manual review of at-risk orders.
- **Prioritise logistics investment by state** — based on Finding 1, geography is the most actionable, reliable signal identified in this project; the states with the highest late rates (AL, MA, PI) warrant targeted investigation into distribution-hub coverage.

- **Add a seller-level historical late-rate feature in future iterations** — based on Finding 4, order-level features alone carry weak signal; a seller's track record is a plausible proxy for operational risk not otherwise captured in this dataset.
- **Enrich the dataset with carrier tracking, weather, and warehouse dispatch-time data before expecting materially better precision** — based on Finding 3 and the interpretation in Section 4.6, the current feature set has a low performance ceiling regardless of model choice.

5.3 Project Limitations

- The severe class imbalance (92:8) limits any model's precision at a usable recall; SMOTE addresses training-set balance but not the underlying information gap in the available features.
- The ROC-AUC ceiling of ~0.56–0.61 across all five models suggests this feature set has an inherent performance ceiling that further hyperparameter tuning is unlikely to substantially raise.
- The KNN elbow search (used to select $K = 11$) was tuned on SMOTE-resampled data rather than isolated strictly to training folds — a mild leakage risk noted for future iterations, though it did not affect the other four models' evaluation.
- A feature-name/column-index mismatch was identified in an earlier version of the feature-importance and decision-tree visualisations (pre-encoding feature names had been paired with post-one-hot-encoding importances); this was corrected during final quality review, and the corrected code is reflected in this report's results.

5.4 Future Improvements

- Explore cost-sensitive learning (e.g. focal loss or class-weighted objectives) alongside or instead of SMOTE.
- Merge seller-level historical on-time-delivery-rate as a leakage-safe proxy for operational risk.
- Enrich the dataset with external data: weather, public holidays, and carrier tracking events.
- Replace one-hot encoding of state features with target/frequency encoding to reduce the 63-column dimensionality and produce more interpretable feature-importance rankings.

5.5 Conclusion

This project delivered a complete, leakage-free analytics pipeline — from the raw Olist relational tables, through exploratory data analysis and feature engineering, to five statistically compared classifiers — in order to predict late deliveries on a major Brazilian e-commerce platform. The most actionable finding to emerge is geographic: late-delivery rates triple across Brazilian states, offering a clear, concrete lever for operational improvement that does not depend on further modelling work. The machine learning models themselves confirm genuine but modest predictive signal (ROC-AUC $\approx$

0.61 at best) rather than a strong classifier, which correctly reflects that order-level data alone cannot fully explain delivery lateness. Rather than viewing this as a shortfall, the project treats it as a well-evidenced conclusion: richer, external logistics data — not further algorithmic tuning — is the natural next step toward a materially more precise late-delivery predictor.

REFERENCES

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>

Küp, B. Ü., Küp, E. T., & Koçak, G. (2025). Real-time prediction of delivery delay in supply chains using machine learning approaches [Preprint]. SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5054862

Lesmana et al. (2025). Machine learning implementation for e-commerce delivery delay prediction using XGBoost algorithm. *Green Engineering: Journal of Engineering and Applied Science*, 2. <https://engineering.ifrel.org/index.php/GreenEngineering/article/download/219/47>

Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Apache Software Foundation. (2024). Apache Airflow documentation (Version 2.9.2). <https://airflow.apache.org/docs/>

PostgreSQL Global Development Group. (2024). PostgreSQL 18 documentation. <https://www.postgresql.org/docs/18/>